

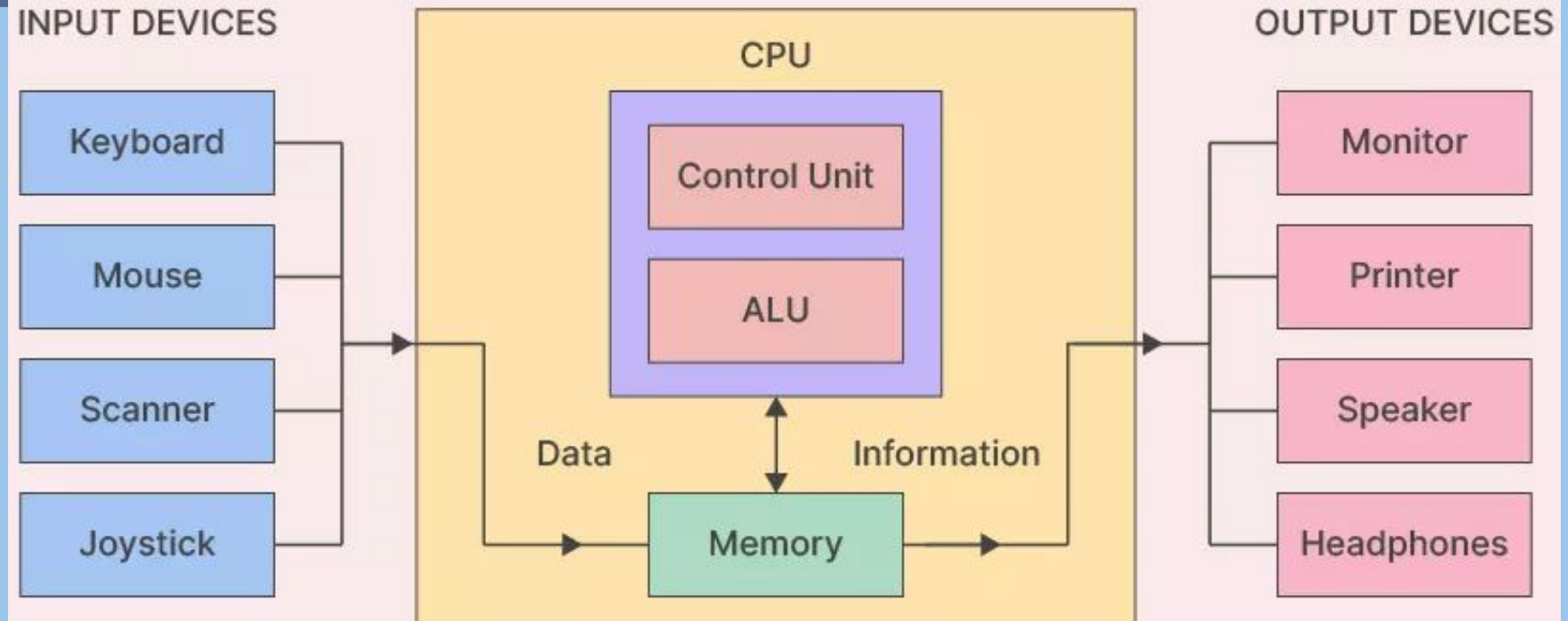


Introduction to Coding (With Python!)

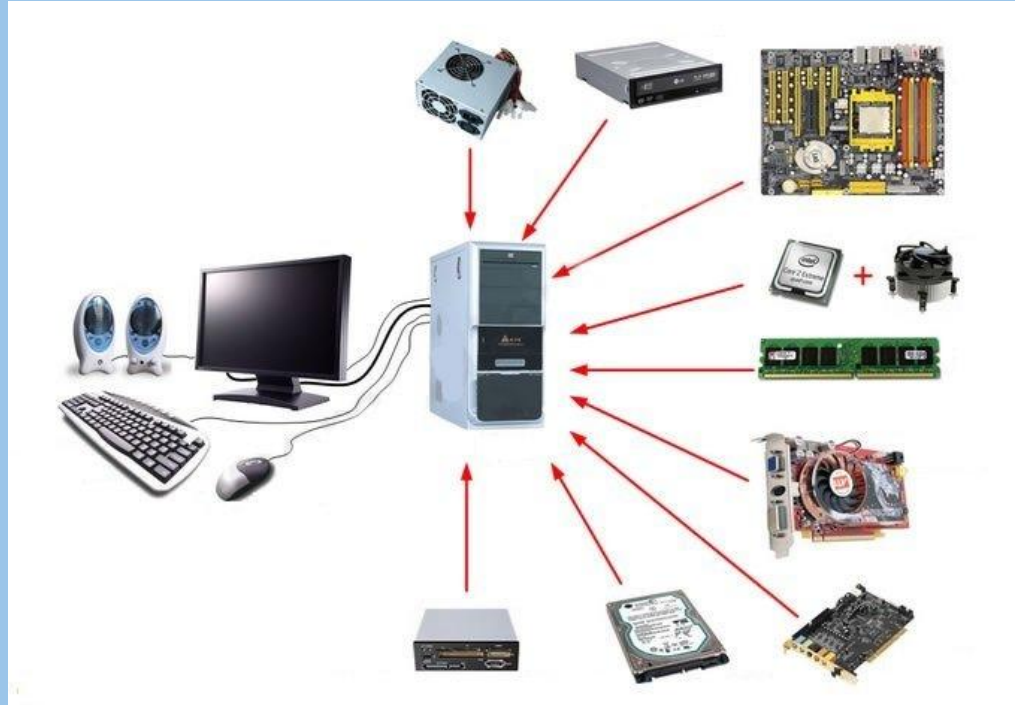
Fermilab - FNAL - Class 1

[illegible]

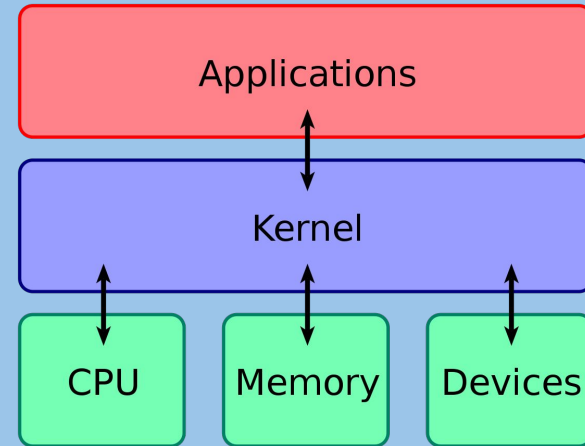
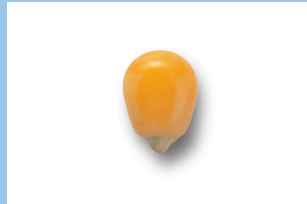
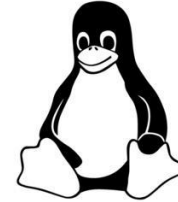
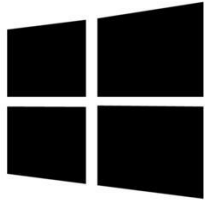
Computer Architecture



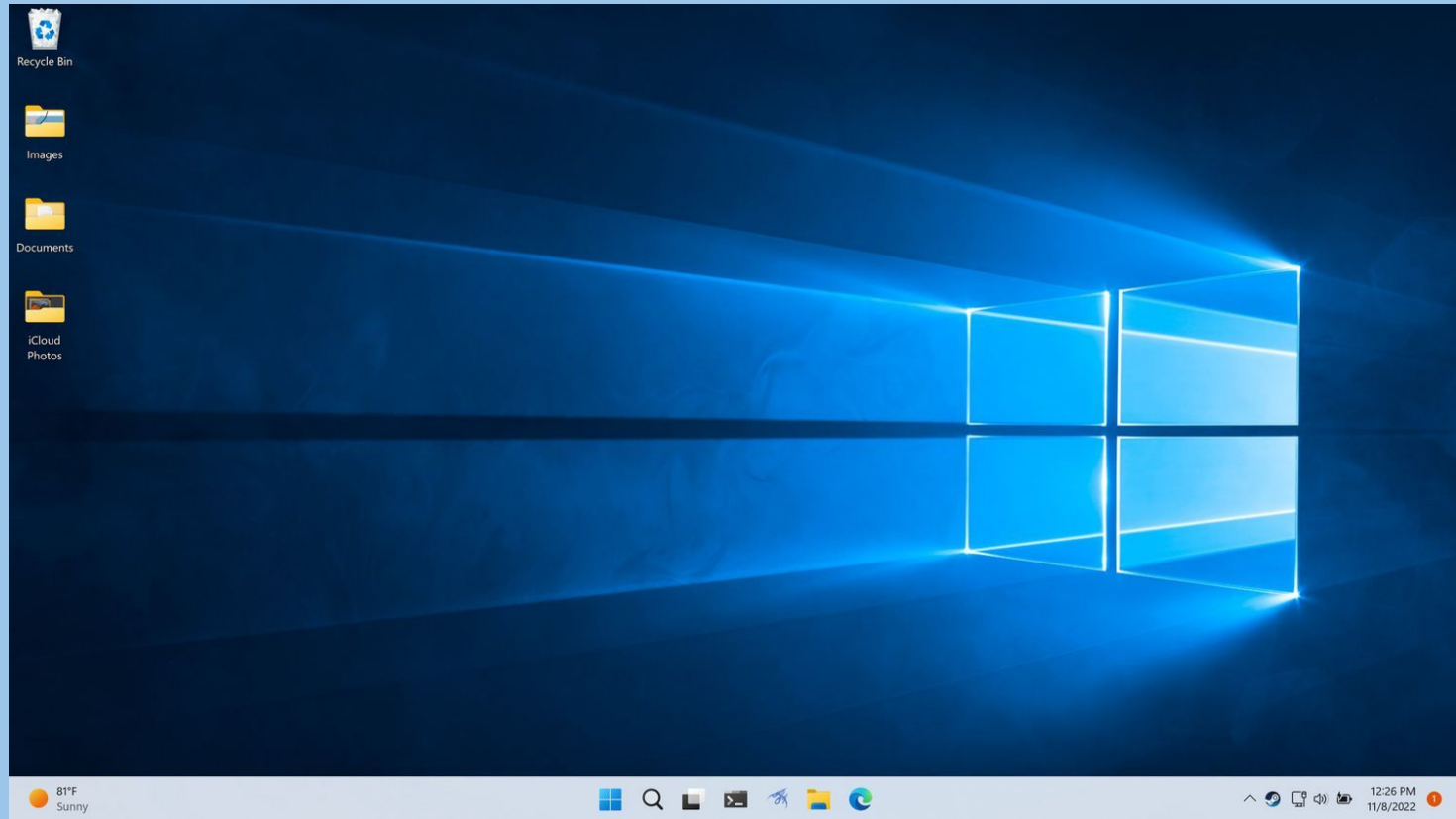
Hardware



Operating System (OS) and Kernel



Graphical User Interface (GUI)



Command Line Interface (CLI)

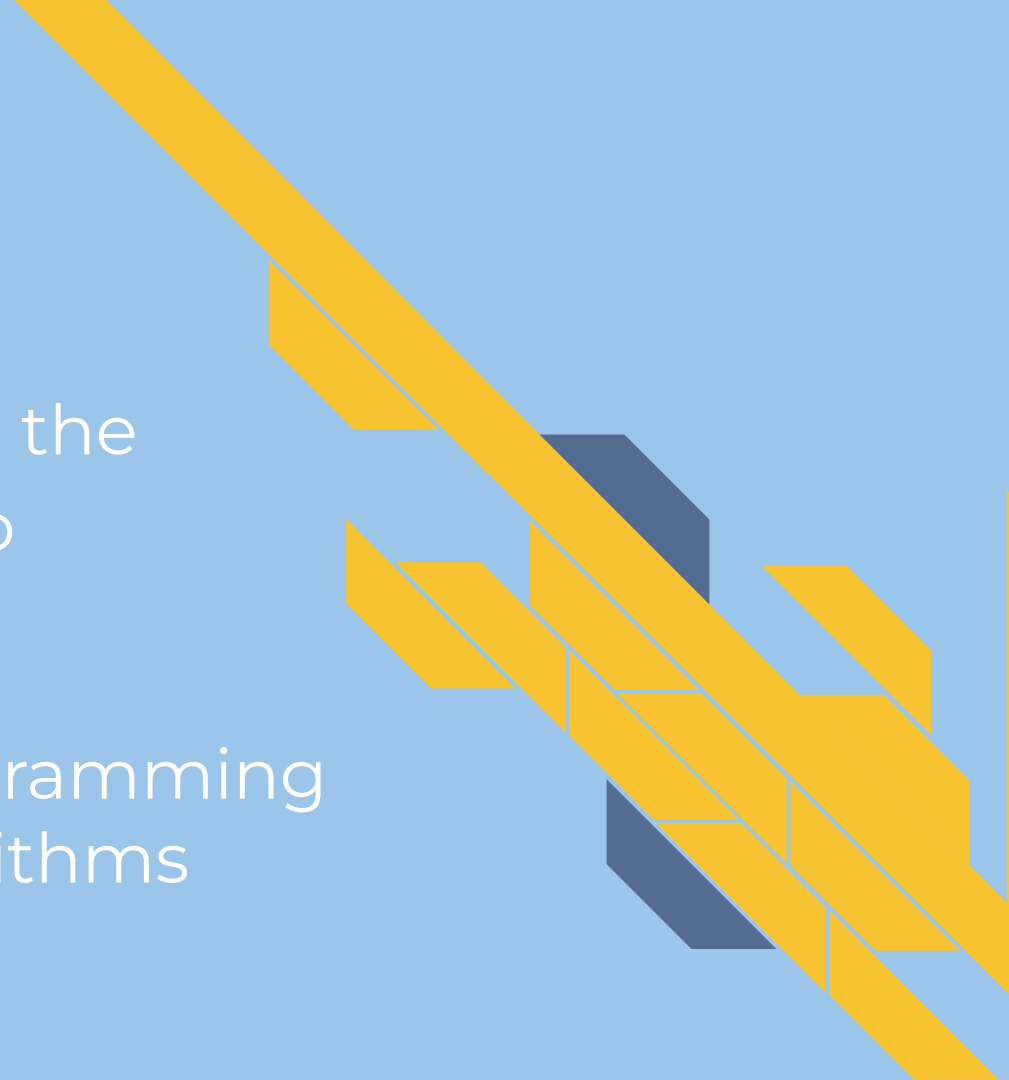
```
[root@localhost ~]# ping -q fa.wikipedia.org
PING text.pmtpa.wikimedia.org (208.80.152.2) 56(84) bytes of data.
^C
--- text.pmtpa.wikimedia.org ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 540.528/540.528/540.528/0.000 ms
[root@localhost ~]# pwd
/root
[root@localhost ~]# cd /var
[root@localhost var]# ls -la
total 72
drwxr-xr-x. 18 root root 4096 Jul 30 22:43 .
drwxr-xr-x. 23 root root 4096 Sep 14 20:42 ..
drwxr-xr-x. 2 root root 4096 May 14 00:15 account
drwxr-xr-x. 11 root root 4096 Jul 31 22:26 cache
drwxr-xr-x. 3 root root 4096 May 18 16:03 db
drwxr-xr-x. 3 root root 4096 May 18 16:03 empty
drwxr-xr-x. 2 root root 4096 May 18 16:03 games
drwxrwx--T. 2 root gdm 4096 Jun 2 18:39 gdm
drwxr-xr-x. 38 root root 4096 May 18 16:03 lib
drwxr-xr-x. 2 root root 4096 May 18 16:03 local
lrwxrwxrwx. 1 root root 11 May 14 00:12 lock -> ../run/lock
drwxr-xr-x. 14 root root 4096 Sep 14 20:42 log
lrwxrwxrwx. 1 root root 10 Jul 30 22:43 mail -> spool/mail
drwxr-xr-x. 2 root root 4096 May 18 16:03 nis
drwxr-xr-x. 2 root root 4096 May 18 16:03 opt
drwxr-xr-x. 2 root root 4096 May 18 16:03 preserve
drwxr-xr-x. 2 root root 4096 Jul 1 22:11 report
lrwxrwxrwx. 1 root root 6 May 14 00:12 run -> ../run
drwxr-xr-x. 14 root root 4096 May 18 16:03 spool
drwxrwxrwt. 4 root root 4096 Sep 12 23:50 tmp
drwxr-xr-x. 2 root root 4096 May 18 16:03 yp
[root@localhost var]# yum search wiki
Loaded plugins: langpacks, presto, refresh-packagekit, remove-with-leaves
rpmfusion-free-updates | 2.7 kB | 00:00
rpmfusion-free-updates/primary_db | 206 kB | 00:04
rpmfusion-nonfree-updates | 2.7 kB | 00:00
updates/metalink | 5.9 kB | 00:00
updates | 4.7 kB | 00:00
updates/primary_db 73% [=====] ] 62 kB/s | 2.6 MB | 00:15 ETA
```



What is **Code**?

Code is a set of instructions given to the computer in order to perform a task.

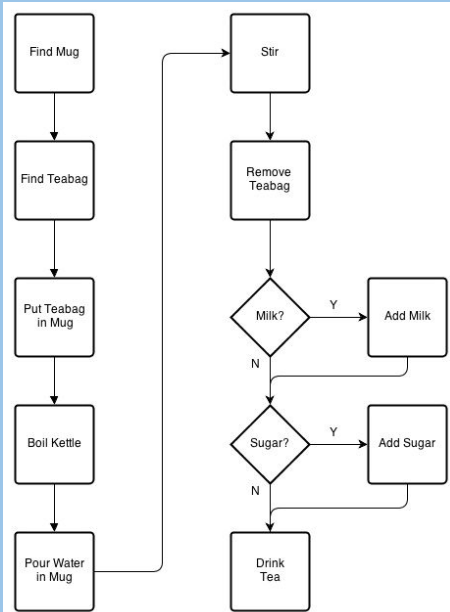
Code is written in a programming language through algorithms





Algorithms are self contained step by step sets of instructions that must be performed **in order** to solve a specific problem or a group of problems.

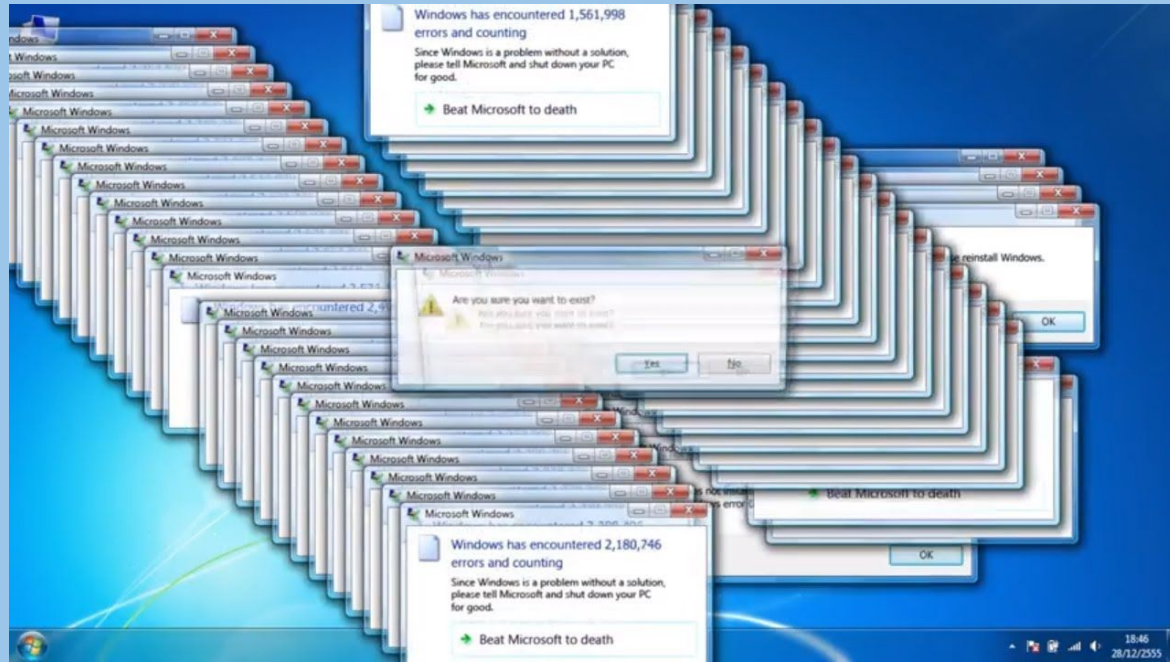
We use algorithms everywhere! Take this algorithm for making tea for example!



```
Start
| Find Mug
| Find Teabag
| Put Teabag in Mug
| Boil Keettle
| Pour Water in Mug
| Stir
| Remove Teabag
| If (Milk?)
|   | Add Milk
| End If
| If (Sugar?)
|   | Add Sugar
| End If
| Drink Tea
Stop
```

Tip: See what happens if you switch the steps “Boil Kettle” with “Pour Water in Mug” Does it still work as well?

We use **Algorithms** because computer's aren't so good at messy instructions. It performs everything you tell it to **in order**. So if you give it instructions out of order ...

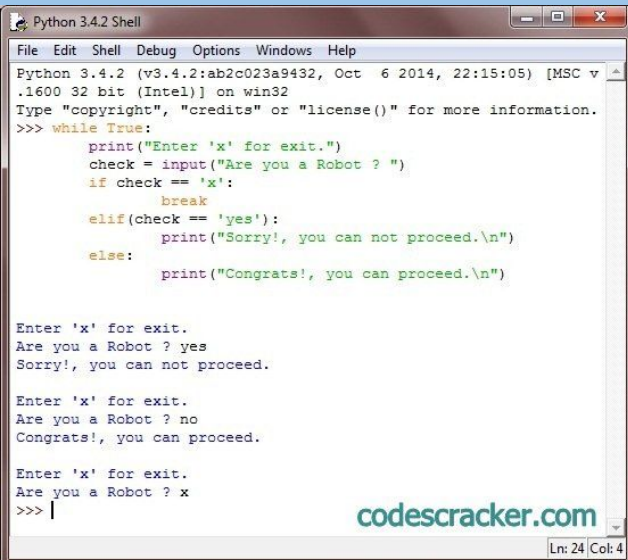


Programming Languages



Code is written with a **Programming Language**. That is, “a set of syntax rules that define how code should be written and formatted.”

Python



```
Python 3.4.2 Shell
File Edit Shell Debug Options Windows Help
Python 3.4.2 (v3.4.2:ab2c023a9432, Oct 6 2014, 22:15:05) [MSC v
.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> while True:
    print("Enter 'x' for exit.")
    check = input("Are you a Robot ? ")
    if check == 'x':
        break
    elif(check == 'yes'):
        print("Sorry!, you can not proceed.\n")
    else:
        print("Congrats!, you can proceed.\n")

Enter 'x' for exit.
Are you a Robot ? yes
Sorry!, you can not proceed.

Enter 'x' for exit.
Are you a Robot ? no
Congrats!, you can proceed.

Enter 'x' for exit.
Are you a Robot ? x
>>>
```

codescracker.com

Ln: 24 Col: 4

Java

```
public static void main(String[] args) {

    int user = 17;

    if (user <= 18) {
        System.out.println("User is 18 or younger");
    }
    else {
        System.out.println("User is older than 18");
    }
}
```

Fortran

```
PROGRAM CABLE
C THIS PROGRAM COMPUTES THE VELOCITY OF A CABLE CAR ON A THOUSAND-FOOT
C CABLE WITH THREE TOWERS
C
C INTEGER TOTDIS, DIST, TOWER
C
C WRITE(*,1)
1 FORMAT('1',9X,'CABLE CAR REPORT'/, ' ', 'DISTANCE',
+ 2X, 'NEAREST TOWER', 2X, 'VELOCITY'/1X, 2X,
+ '(FT)', 19X, '(FT/SEC)')
C
C TOTDIS = 0
C
C 5 IF(TOTDIS.GT.250)GO TO 10
C TOWER = 1
C DIST = TOTDIS
C GO TO 20
C
C 10 IF(TOTDIS.GT.750)GO TO 15
C TOWER = 2
C DIST = IABS(TOTDIS - 500)
C GO TO 20
C
C 15 TOWER = 3
C DIST = 1000 - TOTDIS
C
C 20 IF(DIST.LE.30)VEL = 2.425+0.00175*DIST*DIST
C IF(DIST.GT.30)VEL = 0.625+0.12*DIST-0.00025*DIST*DIST
C
C WRITE(*,40)TOTDIS, TOWER, VEL
C 40 FORMAT(' ',I4,11X,I1,9X,F7.2)
C
C TOTDIS=TOTDIS+10
C
C IF(TOTDIS.LE.1000)GO TO 5
C
C STOP
C END
```



Now that we know what Programming Languages are, ***Why Python?***

1. Python can be used for Pretty Much Anything (Web Design, App Development, Programming, it goes on)
2. It's beginner friendly
3. It's popular and in-demand
4. It's widely used in Data Science
5. All the Big Names use it (IBM, Instagram, Youtube...)
6. Lastly, we use it here at Fermilab!



With that, you're almost ready to begin!
Before you move on though, here are some
coding tips!

Tips

1. Rigor and Formality
2. Abstraction
3. Divide et Impera (Divide, Conquer, and Glue)
4. Modules
5. Models
6. Incremental Steps

1. *Rigor and Formality*

As we said before, computers aren't good with messy instructions but they're *really good* at *clear and precise* ones! Take your time to figure out each step in accomplishing your goal!

My mom said:
"Honey, please go to the market and buy 1 bottle of milk. If they have eggs, bring 6"

I came back with 6 bottles of milk.

She said: "Why the hell did you buy 6 bottles of milk?"

I said: "BECAUSE THEY HAD EGGS!!!!"



2. Abstraction

Code is really useful for representing real objects. Just remember when you're doing this, to isolate what characteristics are important for you.



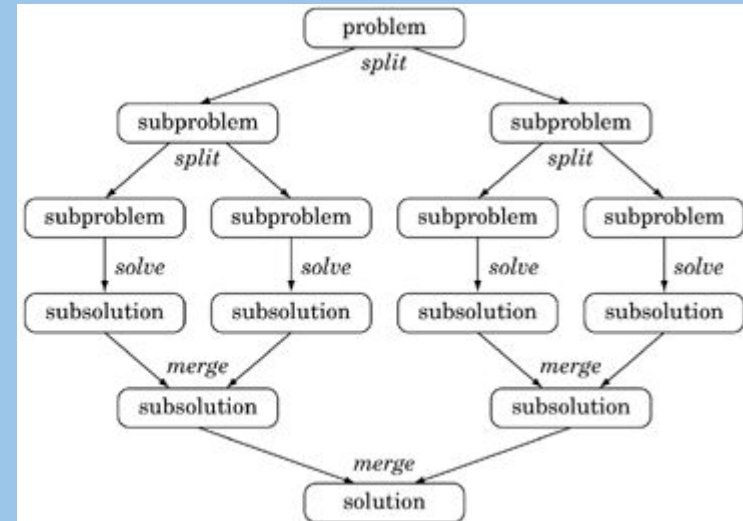
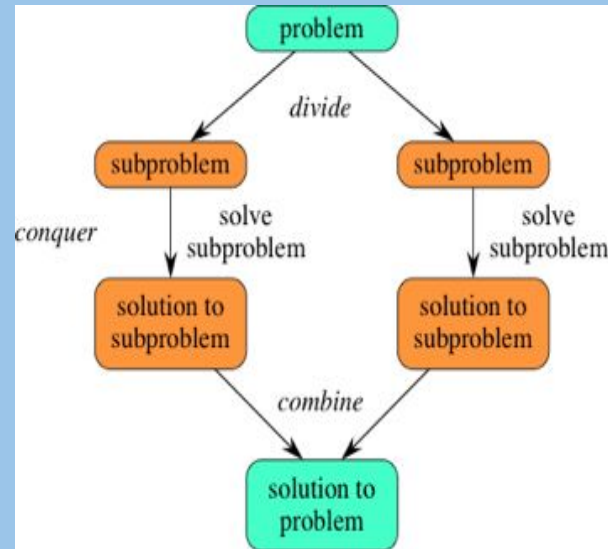
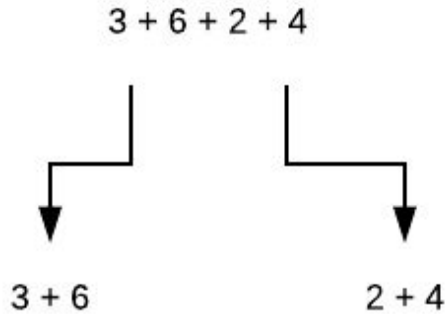
Ex. Cat

Note: Think of some of the key characteristics of these objects:

Person, School,
Room, Student

3. Divide et Impera

A common mistake new programmers make is trying to do all the code at once. It'll benefit you a lot more if you split the overall problem into pieces and solve each piece!



4. Modules

Large systems are built with components called *modules*

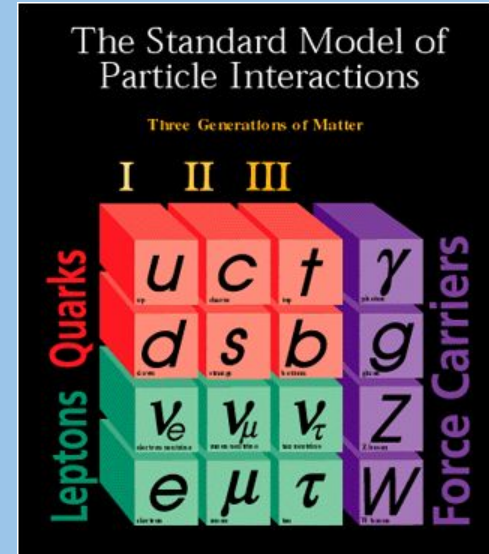
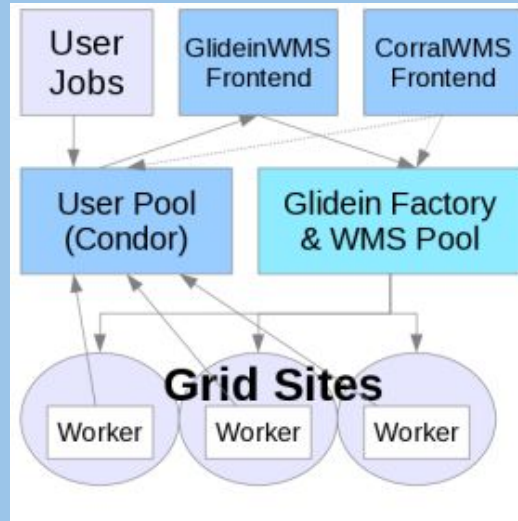
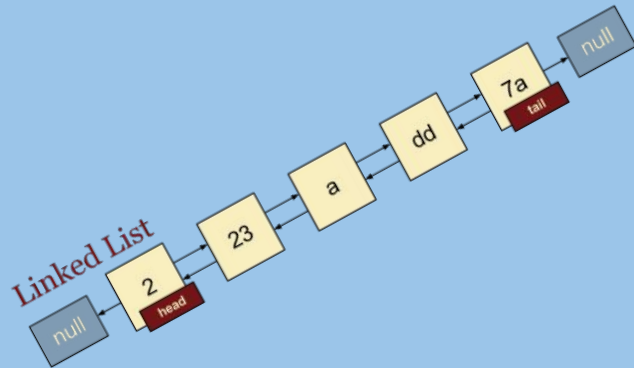
Modules are small bits of code that fit together. For example, you may have one module that checks your bank account, another that deposits money, and another for withdrawing money

Interfaces between *modules* allow to combine them in a mix-and-match way. Real world examples below!



5. Models

Models describe what you do or the interaction and relationship between things. Models are used to create elements that behave like a real system. Even simple models can describe complex artifacts!



6. *Incremental steps*

Take it one small step at the time and plan to expand your program!



Let's code!

```
#include <stdio.h>
int main(void)
{
    int count;

    for (count = 1; count <= 500; count++)
        printf("I will not throw paper airplanes in class.");
    return 0;
}
```

AMEND 10-3





Acknowledgements

Thank you to Jeny Teheran for letting me use some material from her TechSavvy presentation, and to Mirica Yancey to review these slides

Thanks to Pythonbytes, Indeeds, Dice, PYPL for news about Python popularity